



Nodlatch

Human authorization for AI coding agents

AI proposes. Human authorizes. Executor enforces.

A phone-based authorization layer that lets a developer approve or deny exact AI-generated code changes before a separate executor applies them.

iQOO Hackathon 2026 · Developer Tools track · Hyderabad City Battle

Team: Aaditya Singh (lead) · Sasi Chandan · Santhosh Kumar — Malla Reddy University

The problem: one Allow button



Agents act, not just suggest

AI coding agents inspect repositories, edit several files and iterate on their own.



Approval is a single click

The human sees “Allow”. It rarely says which files, which bytes, or under what condition.



Developers pick between two bad options

Babysit every step and lose the benefit of autonomy, or trust the agent too broadly.

Who has this problem: developers and students using coding agents on repositories where an unintended change matters.

Agent wants to make changes

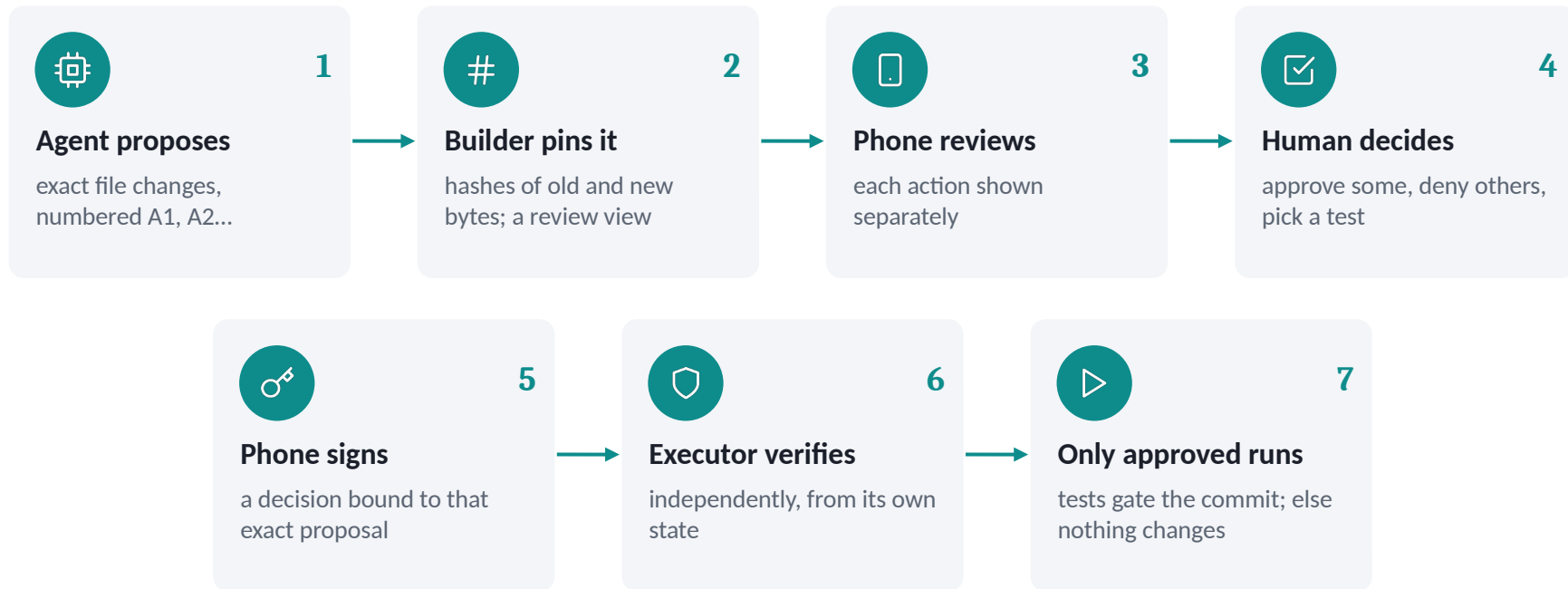
3 files will be modified.

Allow

What exactly did you just authorize?

The core idea

Nodlatch turns a click into an authorization artifact. The executor can only do what that artifact says.



A concrete example

The agent proposes two changes. The developer authorizes one. The executor applies exactly one.

Nodlatch
AI proposes 2 actions

A1
src/auth.py
APPROVED

A2
config.py
DENIED

Authorize A1 only

→
signed decision

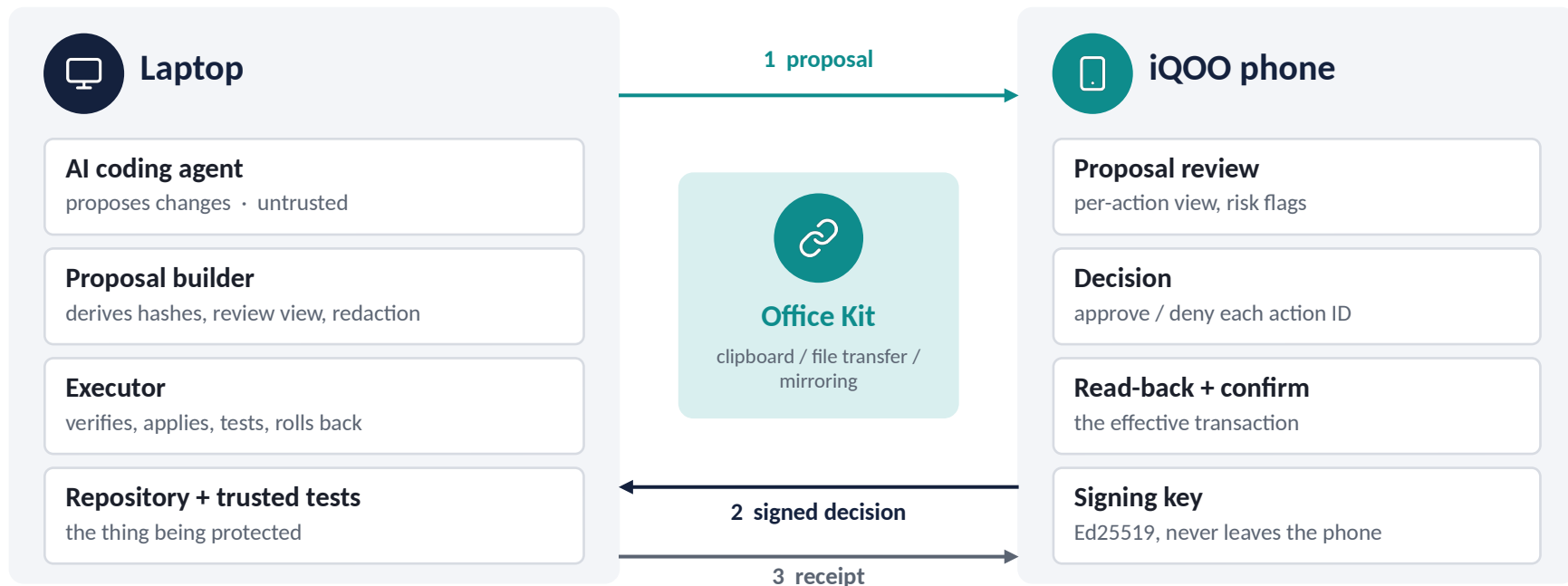


Executor (laptop)

- ✓ Signature valid; decision matches proposal hash
- ✓ A1 new bytes match hash; source file unchanged
- ✓ A1 applied in staging; auth test suite passed
- ✓ A1 written to repository; original bytes backed up
- ✗ A2 never executed: not in the authorized set

Result: APPLIED (A1)

Architecture



Office Kit only transports. The executor trusts nothing that arrives through it: it re-verifies signature, hashes, IDs and expiry from its own stored state.

Why the phone, and what Office Kit does



The phone is the authorization surface

- ✓ A separate device from the one running the agent.
- ✓ Shows requested actions and the effective set the human is signing.
- ✓ Holds the signing key; every decision originates here.
- ✓ Read-back before signing: “You are authorizing A1, not A2, requiring Auth Test Suite v1.”

IF TIME ALLOWS

On-device intent interpreter (typed or spoken → approve/deny selection you confirm), haptics on decision, phone-local user verification, camera-based QR pairing.



Office Kit carries every artifact

proposal

laptop → phone

signed decision

phone → laptop

receipt

laptop → phone

Transport via shared clipboard or file transfer, whichever proves reliable on the real devices. Screen mirroring shows the phone during the demo.

Office Kit is not the security boundary. A payload is valid because the executor verified it, not because of the channel it came through.

Integrity checks and the test gate



Signature

Ed25519, phone-held key



File hashes

new bytes match what was reviewed



Protected paths

.env*, keys, test infrastructure



Single-use + expiry

replay and late use rejected



Proposal hash

decision bound to exact proposal



Source recheck

target unchanged since proposal



Default deny

unknown, malformed, stale → reject



Exact backups

original bytes kept for rollback

Trusted test gate

Apply approved bytes in a staging copy



Run the trusted suite the executor chose



PASS
write to repo, keep
backups

FAIL
real tree untouched

Success means: suite identity matches, exit code 0, and at least the expected number of tests ran. The agent cannot edit the tests that gate its own change.

Tests are an execution condition, not a proof of safety.

Demo: four scenarios in about three minutes

A tiny Python demo repo with an auth module, a small trusted test suite and fake secrets only.



1 Selective approval

Agent proposes A1 (src/auth.py) and A2 (config.py). Approve A1, deny A2. Only A1 is applied; tests pass; receipt on phone.

APPLIED (A1)



2 Tampered artifact

After approval, one character of the proposed file is altered. The executor recomputes the hash and refuses.

REJECTED_ARTIFACT_HASH_MISMATCH



3 Failing test

An authorized change that breaks the auth tests. Staged, tested, never written to the real tree.

TESTS_FAILED_NOT_APPLIED



4 Replay

The same signed decision is submitted a second time. Decisions are single-use.

REJECTED_REPLAY

What is new here, and where the AI sits



The combination is the contribution

Coding agents already have permission modes and hooks; approval services and policy engines (OPA, Cedar) exist. No single element below is claimed as new.

- Phone-side, per-action authorization
- Exact proposal and artifact binding by hash
- Independent executor that trusts neither agent nor channel
- Trusted test as an execution condition
- Office Kit as the phone ↔ laptop workflow



Where the AI is, and is not

AI writes the proposal

A real LLM coding agent produces the new file contents for a task such as “fix this authentication bug”. Planned: a local, open-source model via Ollama, with the phone in the loop.

Planned: on-device intent interpreter

A small on-phone model turns “only the auth change, require the auth tests” into a concrete selection the developer still confirms.

AI never does the security checks

Signature, hash, path and replay checks are deterministic code. Nodlatch is not a chatbot and not a code generator.

MVP, planned implementation, future work



MVP we will demo

- Exact whole-file replacement (modify_file) as the only executable action
- Proposal and action IDs; per-action approve / deny on the phone
- Signed, single-use, time-limited decisions
- Executor verification: signature, hashes, source state, protected paths, default deny
- Trusted test gate, exact-byte backup and rollback
- The four demo scenarios



Planned implementation

- Local / open-source model via Ollama as the coding agent
- Phone authorization app: native Android, Flutter or PWA, locked after first-hour device tests
- Ed25519 signing, exact-byte serialization
- Mediated Python executor, git for traceability, pytest as the trusted suite
- Clipboard and file-transfer adapters over Office Kit
- All Nodlatch code written inside the event window



Future work

- Phone-local user verification bound to signing
- More action types (create, delete, rename)
- Richer, persistent policies
- Stronger isolation between agent and executor
- Signed receipts, pairing and revocation
- IDE and agent integrations, team workflows

What Nodlatch is not

Stated plainly, because a security tool that overclaims is worse than none.



Not an OS sandbox

An agent with the executor's OS privileges could bypass it or its trust anchor. Nodlatch governs actions routed through the executor.



Not a defence against a fully compromised laptop

The design assumes the executor, its trusted key and its replay state are intact.



A signature is not proof of a human

It proves control of a key. Office Kit lets the laptop drive the phone UI; phone-local user verification is planned, not claimed.



Passing tests and matching hashes are not proof of safety

Hashes give integrity and binding; tests are an execution condition. Neither says the code is correct or harmless.



Redaction is not a guarantee

Pattern-based redaction reduces exposure of secrets on the phone; it cannot find every secret.

The team, and why this is buildable in 30 hours



Aaditya Singh

Team lead

B.Tech AI & ML, Malla Reddy University



Sasi Chandan

Team member

B.Tech AI & ML, Malla Reddy University



Santhosh Kumar

Team member

B.Tech AI & ML, Malla Reddy University

Relevant background

- Local LLM in production use: an Ollama-backed chatbot inside the MRU Digital Counselling Management System (web + mobile).
- GuardianDrive: edge-AI drowsiness and distraction detection.
- Co-authored research on audio deepfake detection.
- InnoGenesis 2026 (24-hour national hackathon, Dr. RVR NRI University): built a generative UI engine. ML competition entry.
- Stack: Python, Java, JavaScript, React / Next.js, Node, PyTorch, TensorFlow, Docker.

How we de-risk the build

- Vertical slice first: proposal → phone → decision → one applied change, before Evaluation Round 1.
- Then hashes and source checks, protected paths, test gate and rollback, signing, real LLM agent, polish.
- Written cut list: intent interpreter, second transport, extra demo scenarios go first.
- Backup video, typed fallbacks and a manual import path for the demo.
- Open-source components disclosed: model runtime and model, crypto library, git, pytest, UI framework.